

HeatDraft Pipeline - Cohesive Merged Documentation

Detailed architecture rationale and implementation notes | Generated 2026-02-22 10:52 UTC

Scope

This document combines the WHY and HOW perspectives for the current HeatDraft pipeline, including strict integrity guards, updated visuals, and structured storage.

Navigation

- 1. Input Discovery and File Loading
- 2. Column Normalization and Optional RDKit Enrichment
- 3. Target Conditioning and Hard Range Enforcement
- 4. Leakage Pruning and Split Preconditions
- 5. Category Filtering and Feature Selection Protocol
- 6. Data Structure Integrity Gate
- 7. Target Transform, Weighting, and Objective Design
- 8. Model Tuning, Leaderboard, and Ensemble Construction
- 9. Visualization Overhaul and Diagnostic Depth
- 10. Structured Storage Contract and Reporting
- 11. Operational Modes and Failure Semantics

1. Input Discovery and File Loading

How

- The pipeline resolves the input file from CLI argument first; if missing, it auto-detects a single CSV/XLSX in the working directory.
- It reads data using pandas with the configured header row and immediately checks that the target column exists.
- If target column is missing or file resolution is ambiguous, execution stops with an explicit error instead of guessing.

Why

- Early, deterministic input checks avoid silent schema drift. Most model failures downstream originate from bad assumptions at load time.
- Failing before training starts protects compute budget and ensures every report corresponds to a known schema contract.

Integrity / Validation Checks

- File existence and uniqueness are enforced.
- Target column presence is enforced before any transformation.

Key Outputs

- A normalized in-memory dataframe with a guaranteed target column.

2. Column Normalization and Optional RDKit Enrichment

How

- Column names are normalized to reduce malformed characters and whitespace noise introduced by mixed encodings.
- If a SMILES column exists, RDKit descriptors are computed (molecular weight, logP, TPSA, hydrogen bond descriptors, ring count, rotatable bonds, and fraction sp3).
- Invalid or empty SMILES values remain NaN for descriptor fields and are handled later by the preprocessing stack.

Why

- Model logic depends on exact column names. Encoding and whitespace variation can break downstream selectors or leak features unexpectedly.
- RDKit converts symbolic chemical strings into physically meaningful numeric descriptors that tree models can exploit.

Integrity / Validation Checks

- SMILES enrichment is optional and data-driven; no SMILES means no descriptor branch.

HeatDraft Pipeline - Cohesive Merged Documentation

Detailed architecture rationale and implementation notes | Generated 2026-02-22 10:52 UTC

- Descriptor generation does not mutate target semantics.

Key Outputs

- Expanded feature space when SMILES is present, otherwise baseline schema preserved.

3. Target Conditioning and Hard Range Enforcement

How

- Optional target masking can replace a user-defined target interval with NaN to intentionally exclude uncertain ranges.
- Target is cast to numeric, and rows with missing target are dropped.
- Before logit transform, strict checks enforce no NaN and value range in [0, 100]. Out-of-range targets raise immediately.

Why

- Bounded target semantics are physically meaningful (percentage). Silent clipping hides data quality problems and can corrupt conclusions.
- Explicit failure for invalid targets prevents producing polished reports from invalid scientific assumptions.

Integrity / Validation Checks

- Target mask report includes masked row count.
- Transform guard enforces percentage domain assumptions exactly.

Key Outputs

- Clean target vector compatible with logit/inverse-logit pipeline.

4. Leakage Pruning and Split Preconditions

How

- Known leakage-like columns (number, index, unnamed: 0) are removed before splitting.
- A strict gate checks that both high-threshold and low-threshold classes have at least two rows before train/test split.
- Split is stratified on high/low class when enough high rows exist; otherwise, split proceeds unstratified but still under class-count guard.

Why

- Identifier columns can let trees memorize row identity instead of learning chemistry/process relationships.
- Class preconditions avoid impossible evaluation setups where high-zone metrics cannot be computed.

Integrity / Validation Checks

- RuntimeError when high/low counts are insufficient.
- Deterministic random seed controls split reproducibility.

Key Outputs

- X_train_raw, X_test_raw, y_train_orig, y_test_orig with leakage columns removed.

5. Category Filtering and Feature Selection Protocol

How

- High-cardinality and near-unique categoricals are identified from train data only, then dropped consistently from train/test/full frames.
- Feature selection pipeline (train only) applies sparse-feature drop, near-zero-variance filter, correlation clustering with MI representative selection, and recursive VIF pruning.
- If feature-drop is disabled, a light sparsity keep-rule is still applied to avoid pathological columns.

Why

- Train-only feature selection avoids test leakage while preserving consistent schema propagation.
- Correlation and VIF controls reduce collinearity instability and improve interpretability of feature importance patterns.

HeatDraft Pipeline - Cohesive Merged Documentation

Detailed architecture rationale and implementation notes | Generated 2026-02-22 10:52 UTC

Integrity / Validation Checks

- Dropped categorical columns are validated to exist across all filtered frames.
- Selected column list is propagated in identical order to train/test/full views.

Key Outputs

- Final selected feature matrix and detailed feature-selection report object.

6. Data Structure Integrity Gate

How

- A dedicated validator enforces: non-empty train matrix, no duplicate columns, exact train-test-full column order equality, and index alignment between feature and target objects.
- The validator also enforces zero index overlap between train and test rows.

Why

- Most silent ML bugs are shape/index bugs, not algorithm bugs. Guarding structure immediately after preprocessing catches misuse early.
- Column order mismatches are particularly dangerous with transformed pipelines and can invert model semantics without obvious errors.

Integrity / Validation Checks

- Runtime errors for any mismatch in columns, order, or index alignment.
- Hard failure on train/test overlap.

Key Outputs

- Verified, contract-compliant train/test/full feature tensors for modeling.

7. Target Transform, Weighting, and Objective Design

How

- The percentage target is mapped to logit space to respect $[0, 100]$ boundaries after inverse transform.
- A dynamic high-zone weight ratio is computed from class imbalance and applied during model fitting and CV scoring.
- Objective optimization uses weighted MAE in transformed space with high-zone emphasis.

Why

- The transform stabilizes optimization near hard bounds and prevents impossible >100 or <0 behavior after inverse mapping.
- Weighting aligns optimization with practical goals when high-performing rows are minority but mission-critical.

Integrity / Validation Checks

- High-zone test rows are mandatory; run fails if absent.
- Weight ratio and transformed threshold are logged for auditability.

Key Outputs

- `y_train_trans` and weighted objective context for all model training stages.

8. Model Tuning, Leaderboard, and Ensemble Construction

How

- Optuna tunes `xgboost`, `extra_trees`, `random_forest`, and `hist_gb` under KFold CV using the weighted transformed objective.
- Each tuned model is evaluated on global test and high-zone test subsets.
- Top models feed a `StackingRegressor` with `RidgeCV` meta-learner; optional MoE clones this template per KMeans cluster gate.

Why

- Model diversity plus meta-learning usually outperforms single-architecture assumptions in heterogeneous process datasets.

HeatDraft Pipeline - Cohesive Merged Documentation

Detailed architecture rationale and implementation notes | Generated 2026-02-22 10:52 UTC

- KMeans MoE can specialize experts for different process regimes, but remains optional and strict (no silent fallback).

Integrity / Validation Checks

- Winner selection enforces that a chosen MoE model must exist.
- Metrics table keeps both global and high-zone scores for balanced interpretation.

Key Outputs

- Final metrics leaderboard, winner model object, and best-parameter records.

9. Visualization Overhaul and Diagnostic Depth

How

- The test-set visualization suite now includes: predicted-vs-actual scatter with threshold guides, residual distribution by zone, high-zone model frontier, and largest absolute errors.
- A dedicated calibration curve and calibration table are computed from actual-value quantile bins.
- Low-vs-high feature-gap diagnostics are rendered using a dumbbell view for direct median comparison.

Why

- Single scatter dashboards hide calibration and tail-risk behavior. Multi-view diagnostics make failure modes visible.
- Error concentration and calibration matter more for deployment confidence than a single summary metric.

Integrity / Validation Checks

- Visualization stage requires high-zone rows and low-zone rows in test where relevant diagnostics are computed.
- Optional plots are generated only when their data prerequisites are satisfied.

Key Outputs

- Consistent figure set under the figures directory for fast model review.

10. Structured Storage Contract and Reporting

How

- Artifacts are written into three stable subdirectories under outdir: figures, data, and reports.
- Data exports include test_predictions.csv, metrics_leaderboard.csv, calibration_table.csv, selected_features.csv, low_zone_feature_gaps.csv, and optional vif_table.csv.
- Reports include model_report.json, best_params.json, split_summary.json, and low_zone_diagnostics.json.
- model_report.json contains a nested artifact map so downstream tools can resolve all generated paths without hard-coding.

Why

- Flat output directories become unmanageable as diagnostics grow. Grouping by artifact type improves reproducibility and automation.
- Explicit path maps reduce manual hunting and make integration into dashboards or CI straightforward.

Integrity / Validation Checks

- Directory creation is idempotent.
- Artifact references in reports match actual written files.

Key Outputs

- A machine-readable, automation-friendly output bundle.

11. Operational Modes and Failure Semantics

How

- Dry-run mode executes data-prep and integrity checks, then exits before expensive tuning.
- Full mode trains, evaluates, visualizes, and exports all artifacts.
- The pipeline is intentionally fail-fast: class gates, target bounds, and zone-availability constraints stop execution instead of producing partial misleading reports.

HeatDraft Pipeline - Cohesive Merged Documentation

Detailed architecture rationale and implementation notes | Generated 2026-02-22 10:52 UTC

Why

- In scientific/engineering pipelines, partial success can be dangerous if interpreted as valid evidence.
- Fail-fast behavior makes failure explicit and actionable, reducing hidden debt.

Integrity / Validation Checks

- Dry-run confirms schema and preprocessing viability before long compute jobs.
- Strict runtime errors enforce contract integrity from ingestion to diagnostics.

Key Outputs

- Either a complete trusted artifact package or a clear stopping error with cause.

12. Operational Command Reference

- Run full pipeline: `python heatdraft.py your_file.xlsx --header 0 --outdir outputs`
- Run dry-run only: `python heatdraft.py your_file.xlsx --header 0 --dry-run`
- Disable MoE: `python heatdraft.py your_file.xlsx --disable-moe`
- Regenerate flowchart: `python generate_flowchart.py`
- Regenerate PDFs: `python generate_pdf.py`

13. Output Artifact Reference

- figures/performance_dashboard.png
- figures/calibration_curve.png
- figures/correlation_heatmap.png (optional)
- figures/feature_gaps.png (optional)
- data/test_predictions.csv
- data/metrics_leaderboard.csv
- data/calibration_table.csv
- data/selected_features.csv
- data/low_zone_feature_gaps.csv
- data/vif_table.csv (optional)
- reports/model_report.json
- reports/best_params.json
- reports/split_summary.json
- reports/low_zone_diagnostics.json